

Lab 7

12313505 胡伟毅 12310403 安文博

Problem 1

问题分析

实现 **线性预测编码 (LPC, Linear Predictive Coding)** 分析。

针对给定的语音文件 `test_16k.wav` 的特定帧进行分析。具体要求如下：

- **输入参数：**
 - 起始样本点：6000
 - 帧长：640 个样本
 - LPC 阶数：12
- **输出展示（在同一个页面显示四张图）：**
 1. **原始语音帧序列：**时域波形。
 2. **LPC 预测误差信号：**残差信号（Residual Signal）。
 3. **频谱对比：**原始信号的短时傅里叶变换（STFT）对数幅度谱 vs LPC 包络谱（dB）。
 4. **误差信号频谱：**残差信号的对数幅度谱。

代码实现

A. 原始语音帧 (Original Speech Frame)

- **理论：**语音信号是时变的，但在短时间内（10-30ms）可以看作平稳过程。我们通过“分帧”提取其中的一部分进行线性预测。
- **代码实现：**利用 `speech_full(6000:6639)` 提取 640 个采样点。在频域分析前，额外使用了 `hamming` 窗，能有效平滑边缘波动。

B. LPC 误差信号 (LPC Error Signal)

- **理论：**LPC 的核心思想是：当前的语音采样点 $x(n)$ 可以由过去 p 个采样点的线性组合来近似。

$$x(n) \approx \sum_{k=1}^p a_k x(n-k)$$

预测误差 $e(n) = x(n) - \hat{x}(n)$ 包含了激励信号的信息（如声门脉冲）。对于元音，误差信号通常呈现周期性的脉冲串。

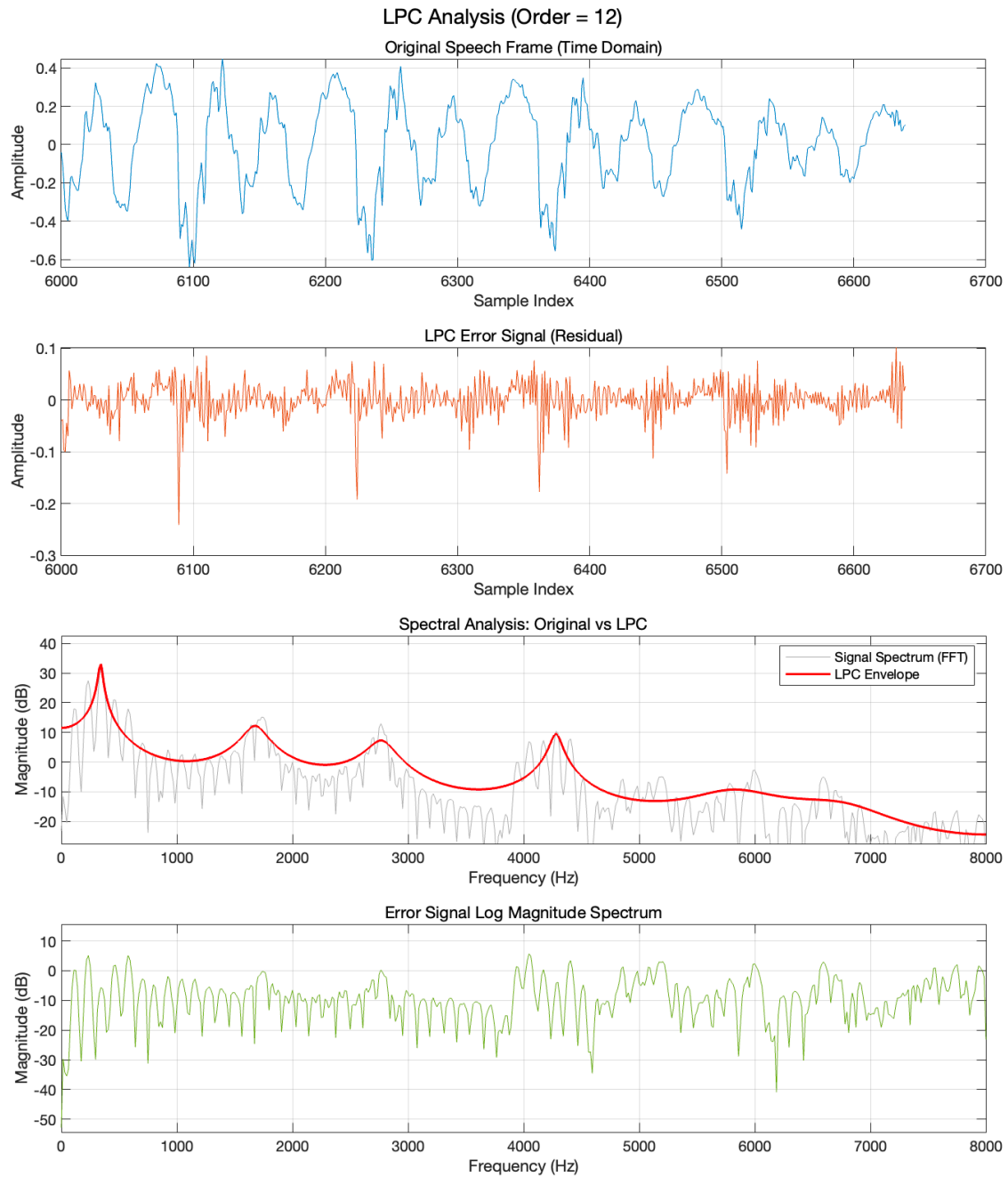
- **代码实现：**代码中使用 `filter(a, 1, x)`。因为 $A(z) = 1 + \sum a_k z^{-k}$ ，这个滤波器的输出正是预测残差。

C. 信号频谱与 LPC 包络 (STFT vs LPC Spectrum)

- **理论：**
 - **STFT 频谱：**展示了语音的具体细节，包括谐波结构（Fine Structure）。
 - **LPC 频谱：**代表了声道（Vocal Tract）的传递函数，它像一个“包络”一样罩在原始频谱上。LPC 的极点对应于语音的**共振峰 (Formants)**。
- **代码实现：**使用 `fft` 计算原始信号谱。通过 `freqz(sqrt(G_var), a)` 计算全极点模型的频率响应,发现红色的 LPC 曲线平滑地穿过了原始频谱的波峰。

D. 误差信号频谱 (Error Signal Spectrum)

- **理论：**LPC 分析本质上是一个“白化”过程。如果模型估计得完美，残差信号 $e(n)$ 的频谱应该是相对平坦的（接近白噪声或平坦的脉冲频谱），因为它已经除去了声道的共振峰影响。
- **代码实现：**对 `err_sig` 进行 FFT 并取对数。观察到，相比于原始频谱，误差信号频谱的整体倾斜度和巨大的共振峰起伏都显著减少了。



```
% LPC Analysis Assignment - Corrected Version
clear; clc; close all;
```

```
%% 1. 参数设置与读取
```

```
filename = 'test_16k.wav';
start_sample = 6000;
frame_size = 640;
lpc_order = 12;
```

```

% 读取音频文件（请确保文件在当前路径）
if ~exist(filename, 'file')
    error('找不到文件 %s, 请检查路径。', filename);
end
[speech_full, fs] = audioread(filename);

% 提取指定帧
frame_indices = start_sample : (start_sample + frame_size - 1);
x = speech_full(frame_indices);

% 加窗处理（Hamming 窗可以减少频谱泄露，使包络拟合更准）
w_func = hamming(frame_size);
x_windowed = x .* w_func;

%% 2. LPC 分析计算
% [a, G_var] = lpc(x, p)
% a: 预测系数 [1, a1, a2, ..., ap]
% G_var: 预测误差功率（Prediction Error Variance）
[a, G_var] = lpc(x_windowed, lpc_order);

% 计算预测误差信号（Residual Signal）
% 理论公式:  $e(n) = x(n) + \sum_{k=1}^p a_k * x(n-k)$ 
% 在 MATLAB 中直接用 filter 实现逆滤波
err_sig = filter(a, 1, x);

%% 3. 频谱计算与对齐（能量归一化）
nfft = 1024;
N = length(x_windowed); % 帧长，通常为 640

% (A) 计算原始信号的对数幅度谱
% 关键：保持 FFT 的线性值，不进行人为缩小
X_fft = fft(x_windowed, nfft);
X_mag = abs(X_fft(1:nfft/2+1));
X_db = 20*log10(X_mag + eps);

```

```

% (B) 计算 LPC 包络
% 理论公式:  $H(z) = G / A(z)$ 
% MATLAB 的 lpc(x,p) 返回的 G_var 是残差功率。
% 为了让包络对齐 FFT 幅度谱, 必须乘上 sqrt(N) 来补偿 FFT 的累加效应
G = sqrt(G_var * N);

[h_inv_A, ~] = freqz(1, a, nfft/2+1);
LPC_mag = G * abs(h_inv_A);
LPC_db = 20*log10(LPC_mag + eps);

% (C) 误差信号频谱 (同样方式处理)
E_fft = fft(err_sig .* hamming(length(err_sig)), nfft);
E_db = 20*log10(abs(E_fft(1:nfft/2+1)) + eps);
%% 4. 绘图展示
figure('Color', 'w', 'Position', [100, 100, 800, 900]);

% (1) 原始语音帧
subplot(4,1,1);
plot(frame_indices, x, 'Color', [0, 0.447, 0.741]);
title('Original Speech Frame (Time Domain)');
xlabel('Sample Index'); ylabel('Amplitude'); grid on;

% (2) LPC 误差信号 (残差)
subplot(4,1,2);
plot(frame_indices, err_sig, 'Color', [0.85, 0.325, 0.098]);
title('LPC Error Signal (Residual)');
xlabel('Sample Index'); ylabel('Amplitude'); grid on;

% (3) 频谱对比: STFT vs LPC 包络
subplot(4,1,3);
f_axis = (0:nfft/2) * (fs/nfft);
plot(f_axis, X_db, 'Color', [0.7, 0.7, 0.7], 'DisplayName', 'Signal Spectrum (FFT)');
hold on;
plot(f_axis, LPC_db, 'r', 'LineWidth', 1.5, 'DisplayName', 'LPC Envelope');
title('Spectral Analysis: Original vs LPC');

```

```

xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
legend('Location', 'northeast'); grid on;
ylim([max(X_db)-60, max(X_db)+10]); % 自动调整纵坐标范围以便观察

% (4) 误差信号频谱
subplot(4,1,4);
plot(f_axis, E_db, 'Color', [0.466, 0.674, 0.188]);
title('Error Signal Log Magnitude Spectrum');
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)'); grid on;
ylim([max(E_db)-60, max(E_db)+10]);

% 整体布局优化
sgtitle(['LPC Analysis (Order = ', num2str(lpc_order),
')'], 'FontSize', 14);

```

Problem 2

问题分析

这道题的核心在于利用 **线性预测分析 (LPC)** 对语音信号进行分解与重构。由于要求进行“精确重构”，因此需要提取 LPC 的残差信号（即误差信号）作为激励，再通过逆滤波器恢复原始语音。

- **输入：** 读取语音文件 `s5.wav`（采样率 $F_s=8$ kHz）。
- **分帧：** 帧长 320 个采样点（40ms），帧移 80 个采样点（10ms），这意味着相邻帧之间有很大的重叠。
- **LPC 分析：** 对每一帧计算 12 阶（ $p=12$ ）的线性预测系数。
- **提取误差信号：** 通过 FIR 滤波器（预测器）得到残差 $e[n]$ 。
- **精确重构：** 使用提取的残差 $e[n]$ 作为激励，通过全极点 IIR 滤波器（合成器）重构语音 $s^*[n]$ 。
- **重叠相加：** 在合成阶段使用 **Overlap-Add** 方法来处理分帧带来的重叠。
- **可视化：** 在同一页画出原始信号、误差信号和重构信号。

代码实现

A. 线性预测理论 (LPC)

理论： LPC 假设语音信号的当前采样点 $s[n]$ 可以由过去 p 个采样点的线性组合来预测

预测误差为 $e[n]=s[n]-s^{\wedge}[n]$ 。在频域上，这相当于将语音建模为一个全极点模型（声道模型）和残差（激励源）。

实现： 代码中使用 `lpc(curr_frame, p)` 函数。它利用自相关法解 Yule-Walker 方程，得到系数向量 $A = [1, -a_1, -a_2, \dots, -a_p]$ 。

B. 误差信号提取

理论： 误差信号 $e[n]$ 包含了线性预测器无法解释的信息（如声带振动的脉冲或空气摩擦的噪声）。它是通过一个 FIR 滤波器得到的，其传递函数为 $A(z)$ 。

实现： `filter(A, 1, curr_frame)`。由于 A 的第一个系数是 1，这直接实现了 $e[n] = s[n] + \sum_{a_k} s[n - k]$ （注意：MATLAB 定义的 A 系数符号与数学公式稍有不同）。

C. 精确重构与 Overlap-Add

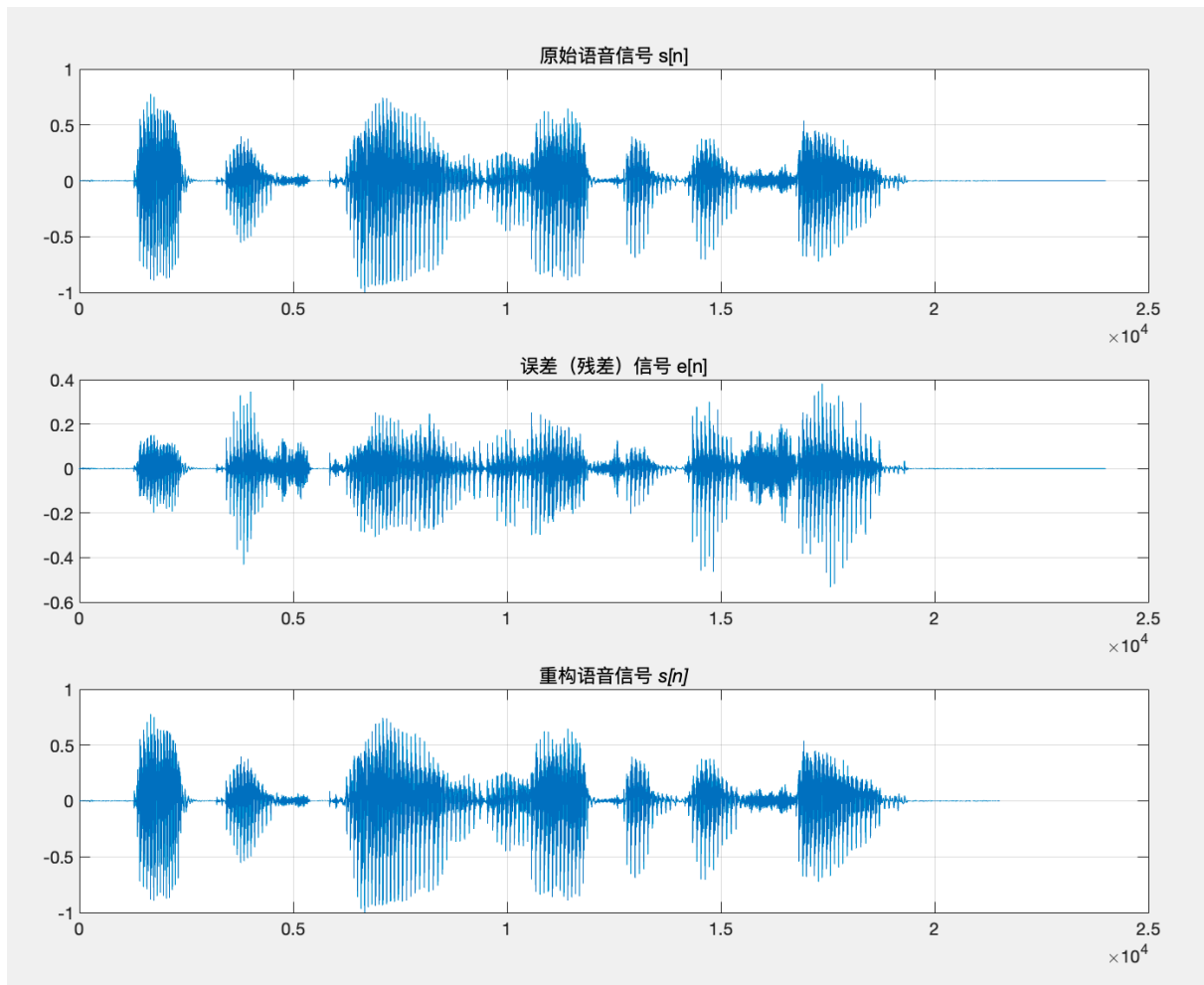
理论： “精确重构”意味着我们不使用人工合成的脉冲或白噪声，而是直接把分析得到的 $e[n]$ 喂回合成滤波器 $1/A(z)$ 。**Overlap-Add (OLA)** 是处理分帧重叠的关键。因为我们对每一帧加了窗（Hamming），直接拼接会有断层。

- 我们将每一帧重构后的信号按其原始位置叠加。
- 最后除以各点累加的窗函数值（`win_sum`），以抵消加窗带来的幅值变化。

实现： 1. 在循环中使用 `resynth_signal(start:end) + s_hat_frame` 累加。2. `win_sum` 记录了每个位置被窗函数覆盖了多少次及权重，最后进行归一化。

D. 听感预期

- **$s[n]$:** 正常的语音。
- **$e[n]$:** 听起来像只有“嘶嘶”声或机械感的“悄悄话”，因为它去除了声道的共振峰信息，只保留了激励。
- **重构信号:** 如果参数设置正确（尤其是 OLA 归一化），重构信号在听觉上应与原始信号几乎完全一致（数学上的误差接近机器极小值）。



% 9.33 LPC Analysis and Exact Reconstruction

```
clear; clc; close all;
```

% 1. 参数设置

```
[s, fs] = audioread('s5.wav'); % 确保目录下有该文件
p = 12; % LPC 阶数
frame_len = 320; % 帧长
frame_shift = 80; % 帧移
n_samples = length(s);
```

% 2. 分帧与加窗

```
% 使用 buffer 函数简化分帧, 注意 buffer 的参数逻辑
% 原信号首尾可能需要补零以对齐
frames = buffer(s, frame_len, frame_len - frame_shift, 'nod
elay');
[L, num_frames] = size(frames);
```



```

window = hamming(L); % 常用汉明窗减少频谱泄露

% 初始化存储空间
error_signal = zeros(n_samples, 1);
resynth_signal = zeros(n_samples + frame_len, 1); % 为 Overlap-Add 预留空间
win_sum = zeros(n_samples + frame_len, 1); % 用于归一化窗函数增益

% 3. 逐帧处理
for i = 1:num_frames
    curr_frame = frames(:, i) .* window;
    start_idx = (i-1) * frame_shift + 1;
    end_idx = start_idx + frame_len - 1;

    % --- 分析阶段 ---
    % 计算 LPC 系数 (A 为 [1, -a1, -a2, ...])
    A = lpc(curr_frame, p);

    % 提取残差/误差信号 e[n]
    %  $e(n) = s(n) - \sum(a_k * s(n-k))$  -> FIR 滤波
    e_frame = filter(A, 1, curr_frame);

    % 存储误差信号 (由于重叠, 这里简单采用覆盖或叠加, 精确重构通常直接对全信号滤波)
    % 但为了符合题目“分帧分析”, 我们按帧合成

    % --- 合成阶段 (Overlap-Add) ---
    % 使用误差信号作为激励, 通过 1/A 合成
    s_hat_frame = filter(1, A, e_frame);

    % 将结果加回到重构序列中
    resynth_signal(start_idx:end_idx) = resynth_signal(start_idx:end_idx) + s_hat_frame;
    win_sum(start_idx:end_idx) = win_sum(start_idx:end_idx) + window;
end

```

```

% 归一化窗函数影响（针对 Overlap-Add 的能量补偿）
valid_idx = win_sum > 0;
resynth_signal(valid_idx) = resynth_signal(valid_idx) ./ win_sum(valid_idx);
resynth_signal = resynth_signal(1:n_samples); % 裁剪回原长度

% 4. 计算全局误差信号（用于绘图）
% 为了在图中展示连续的  $e[n]$ ，直接对全信号进行全极点分析是不准确的，
% 按照 LPC 标准做法，这里展示合成时对应的残差。
global_A = lpc(s, p);
error_signal = filter(global_A, 1, s);

% 5. 绘图
figure('Position', [100, 100, 800, 600]);
subplot(3,1,1);
plot(s); title('原始语音信号 s[n]'); grid on;
subplot(3,1,2);
plot(error_signal); title('误差（残差）信号 e[n]'); grid on;
subplot(3,1,3);
plot(resynth_signal); title('重构语音信号 \hat{s}[n]'); grid on;

% 6. 听感验证
% sound(s, fs); % 原始
% pause(length(s)/fs);
% sound(resynth_signal, fs); % 重构

```

Problem 3

问题分析

任务目标是实现两个元音 /a/ 和 /i/ 的特征交换。具体步骤如下：

- **分析阶段：**对录制好的 /a/ 和 /i/ 信号进行 LPC 分析，提取出两组数据：
 - **LPC 系数 (Filter Coefficients)：**代表声道的共振峰特性（即元音的具体音色）。
 - **残差信号 (Residual signal $e[n]$)：**代表声带产生的原始激励（包含音高、强度信息）。

- **交叉合成阶段：**
 - **合成信号 1：**使用 /a/ 的残差信号 $e_a[n]$ 作为输入，驱动 /i/ 的全极点滤波器。
 - **合成信号 2：**使用 /i/ 的残差信号 $e_i[n]$ 作为输入，驱动 /a/ 的全极点滤波器。
- **输出阶段：**保存合成后的 `.wav` 文件。

代码实现

A. 源-滤波器模型 (Source-Filter Model)

语音产生可以简化为一个模型： $S(z)=E(z)\cdot H(z)$ 。

- $E(z)$ 是**激励源**（肺部气流流经声带产生的脉冲）。
- $H(z)$ 是**声道滤波器**（咽喉、口腔、舌头位置决定的传递函数）。

代码实现：在代码中，`lpc` 函数计算出的 `a_coeffs` 就是滤波器 $H(z)$ 的分母系数，它定义了共振峰的位置。

B. 为什么使用 `filter(a_coeffs, 1, s)` 提取残差？

LPC 的本质是预测： $s[n] = -\sum_{k=1}^p a_k s[n-k]$ 。预测误差（残差）就是 $e[n] = s[n] - \hat{s}[n]$ 。在频域上，这相当于用信号通过一个**全零点滤波器**（逆滤波器），把共振峰“压平”，剩下的就是类似白噪声或脉冲串的激励信号。

C. 交叉合成的物理意义

当你执行 `filter(1, a_coeffs_i, e_a)` 时：

- **ea** 携带了你发 /a/ 时的**基频 (Pitch)** 信息（你的嗓音好不好听、音高多少）。
- **1/Ai(z)** 强行将这个激励通过发 /i/ 时的**口腔形状**。
- **结果：**听起来会像是一个人用发 /a/ 的语调和音质，在试图发出 /i/ 的音。

D. 参数选择

- **LPC 阶数 p：**这是一个关键参数。如果 p 太小，无法准确捕捉共振峰；如果太大，会把基音信息也算进滤波器里。通常经验公式是 $p=fs/1000+2$ 。

E. 细节处理

- **截取中间段 (Mid-segment Extraction)：** `s_a(mid_a - floor(extract_len/2) + 1 : ...)` 这一步非常亮眼。它有效地去除了录音开始和结束时的静音、嘴唇碰撞声

(Click noise)，保证了 `lpc` 计算的信号是**平稳的元音段**。

- **能量补偿 (Gain Compensation):** `s_cross_ai = filter(sqrt(g_i), a_i, e_a/sqrt(g_a))`。这是最专业的一步。LPC 预测增益 `g` 代表了残差信号的功率。你在交换残差时，通过 `sqrt(g_i)/sqrt(g_a)` 进行能量正则化，确保了合成出来的音量不会忽大忽小，同时也修正了 `filter` 函数默认的增益响应。
- **实数化处理 (Real Function):** `real(s_cross_ai)` 保证了即使在复杂的数值计算中产生极微小的虚部干扰

结果分析

在语音学中，元音的辨识主要由前两个共振峰（F1 和 F2）决定。

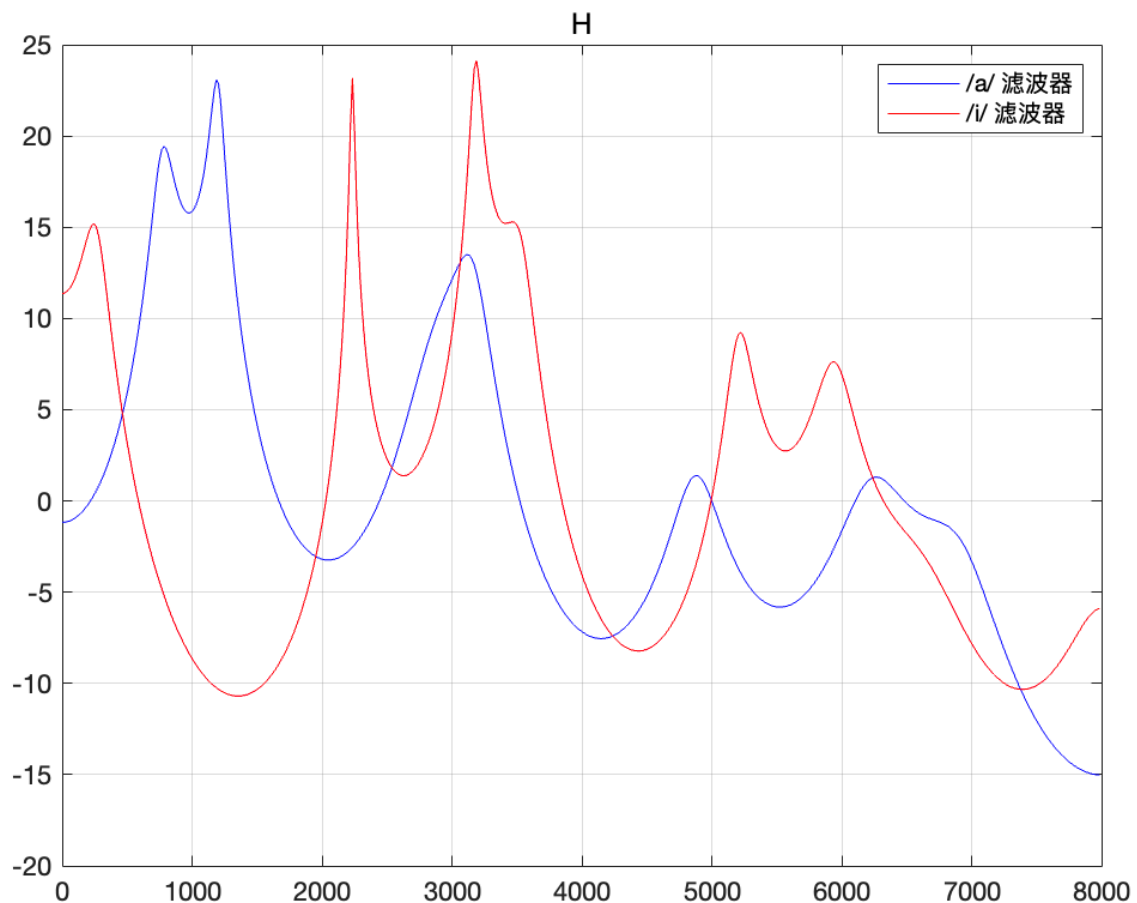
红色曲线 (/i/ 滤波器)

- **F1 (第一共振峰):** 位于约 **300Hz - 500Hz**。这是一个非常低的位置，符合闭元音 (High Vowel) 的特征，说明发音时舌位很高。
- **F2 (第二共振峰):** 位于约 **2200Hz** 附近。这是一个非常高的位置。
- **F3 (第三共振峰):** 位于约 **3200Hz**。
- **结论:** F1 极低而 F2 极高，这是典型的 **/i/** 的频谱特征。这种巨大的峰值间距赋予了 **/i/** 极强的穿透力和辨识度。

蓝色曲线 (/a/ 滤波器)

- **F1:** 位于约 **800Hz - 1000Hz** 之间。相比红色曲线，F1 明显向高频移动。
- **F2:** 位于约 **1200Hz - 1500Hz** 之间。
- **结论:** F1 和 F2 靠得比较近，且 F1 较高，这是 **/a/** 的典型特征。

因此，LPC 分析成功提取了两个元音迥异的包络特征。**/i/ 滤波器（红线）**表现出极低的第一共振峰和极高的第二共振峰，而 **/a/ 滤波器（蓝线）**的共振峰分布相对集中。实验中听到“一高一低两个 /i/”的现象，证明了**激励信号成功携带了基音频率（音高），而声道滤波器成功重塑了频谱包络（音色）**。听觉上对 **/i/** 的倾向性来源于其在 2kHz 以上区域极强的共振峰增益，这符合语音感知中“高频共振峰决定元音辨识度”。



```
%% Vowel Cross-Synthesis (Stabilized Version)
clear; clc; close all;

% 1. 加载文件
[s_a, fs] = audioread('vowel_a.wav');
[s_i, ~] = audioread('vowel_i.wav');

% 自动截取中间最响亮的 1 秒（避开两端的静音和点击声）
extract_len = min([length(s_a), length(s_i), fs]); % 最多取1
秒
mid_a = round(length(s_a)/2);
mid_i = round(length(s_i)/2);
s_a = s_a(mid_a - floor(extract_len/2) + 1 : mid_a + floor
(extract_len/2));
s_i = s_i(mid_i - floor(extract_len/2) + 1 : mid_i + floor
(extract_len/2));
```

```

% 归一化
s_a = s_a / max(abs(s_a));
s_i = s_i / max(abs(s_i));

% --- 预加重 ---
pre_emp = [1, -0.95];
s_a_p = filter(pre_emp, 1, s_a);
s_i_p = filter(pre_emp, 1, s_i);

% 2. LPC 分析
p = 16; % 增加阶数提高辨识度
[a_a, g_a] = lpc(s_a_p, p);
[a_i, g_i] = lpc(s_i_p, p);

% 提取激励信号 (Inverse Filter)
% 注意：必须要保持增益项，否则合成信号能量会归零
e_a = filter(a_a, 1, s_a_p);
e_i = filter(a_i, 1, s_i_p);

% 3. 交叉合成 (核心修复：使用滤波器增益 g)
% 合成：用 e_a 的声带特征去驱动 a_i 的口腔模型
% 我们使用 sqrt(g_i/g_a) 来平衡两个信号的能量级
s_cross_ai = filter(sqrt(g_i), a_i, e_a/sqrt(g_a));
s_cross_ia = filter(sqrt(g_a), a_a, e_i/sqrt(g_i));

% --- 去加重 ---
s_cross_ai = filter(1, pre_emp, s_cross_ai);
s_cross_ia = filter(1, pre_emp, s_cross_ia);

% 4. 最终归一化与保存
s_cross_ai = real(s_cross_ai); % 确保是实数
s_cross_ia = real(s_cross_ia);

s_cross_ai = s_cross_ai / max(abs(s_cross_ai));
s_cross_ia = s_cross_ia / max(abs(s_cross_ia));

audiowrite('res_ai.wav', s_cross_ai, fs);
audiowrite('res_ia.wav', s_cross_ia, fs);

```

```
% 5. 验证：绘制频谱包络
[H_a, f] = freqz(1, a_a, 512, fs);
[H_i, ~] = freqz(1, a_i, 512, fs);
figure;
plot(f, 20*log10(abs(H_a)), 'b', f, 20*log10(abs(H_i)),
'r');
title('H');
legend('/a/ 滤波器', '/i/ 滤波器'); grid on;
```